

完整软件过程定义文档

Dyad 软件过程定义文档

1. 过程概述

Dyad 是一个本地运行的开源 AI 应用构建工具，采用 Electron、React、TypeScript 技术栈。项目面向希望快速构建应用原型的开发者和 AI 应用使用者，支持用户通过自然语言描述生成应用原型，并围绕界面、代码、交互、本地预览、配置管理和发布准备进行持续修改。

Dyad 的软件过程采用轻量化、迭代式、可验证的工程过程。它保留需求、设计、实现、测试、发布、支持和过程改进等基本活动，但不采用重文档、重审批的流程。过程的重点是让每个阶段都有明确输入、活动、输出和质量检查，并在适合的节点引入 AI 工具提高效率。AI 不替代需求确认、架构决策、代码合并和发布判断，只承担草案生成、上下文分析、测试补充、日志摘要和风险提示等辅助任务。

本过程适用于 Dyad 的功能迭代、缺陷修复、发布准备和过程复盘。每一轮迭代以需求项或问题项为单位推进，经过需求澄清、设计确认、工程实现、验证确认、发布交付和反馈改进后闭环。对于探索性功能，可以先生成原型并进行快速验证；对于文件系统访问、模型调用、项目生成、打包发布等高风险功能，需要保留更严格的评审和测试门禁。

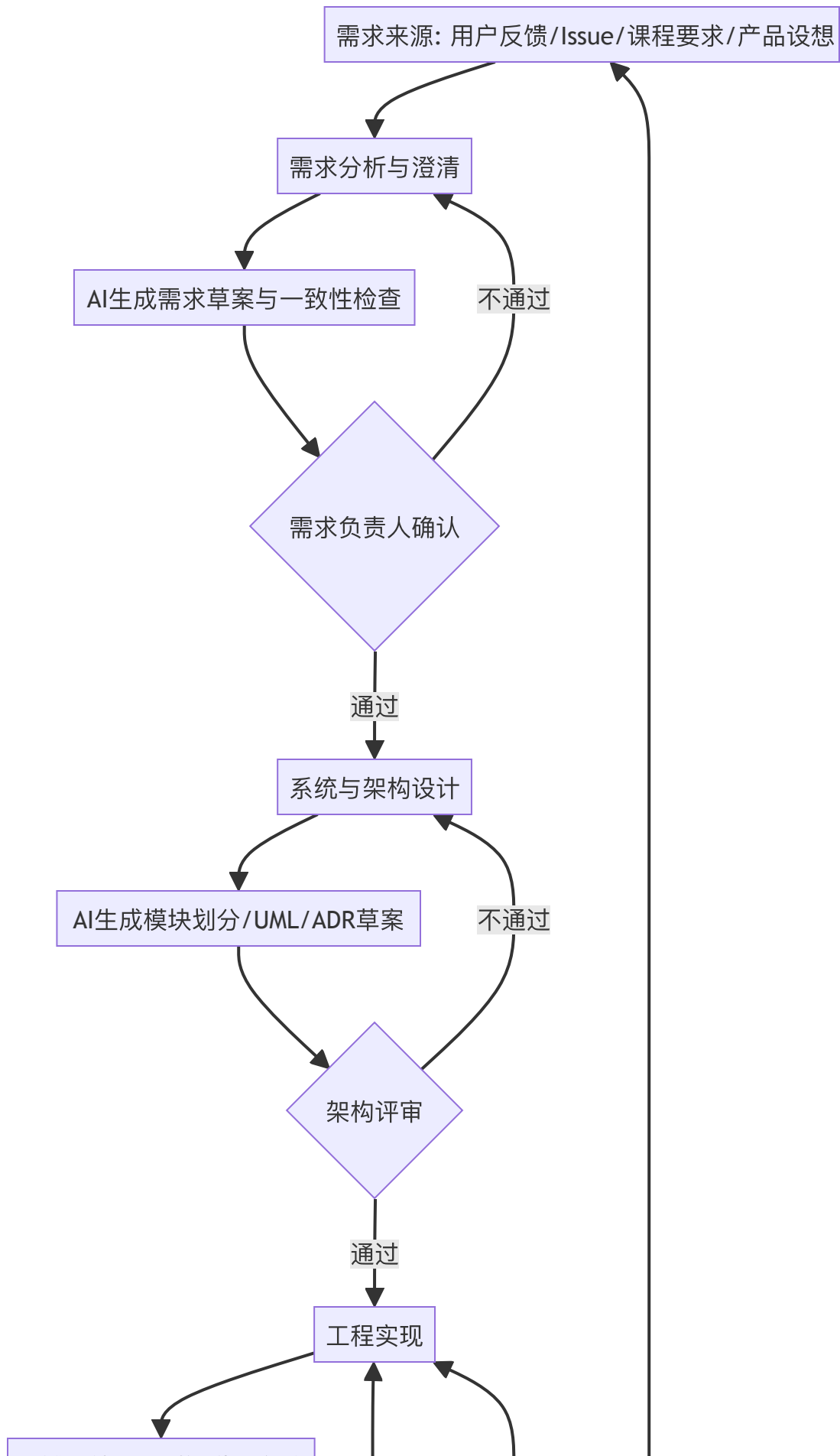
2. 过程目标

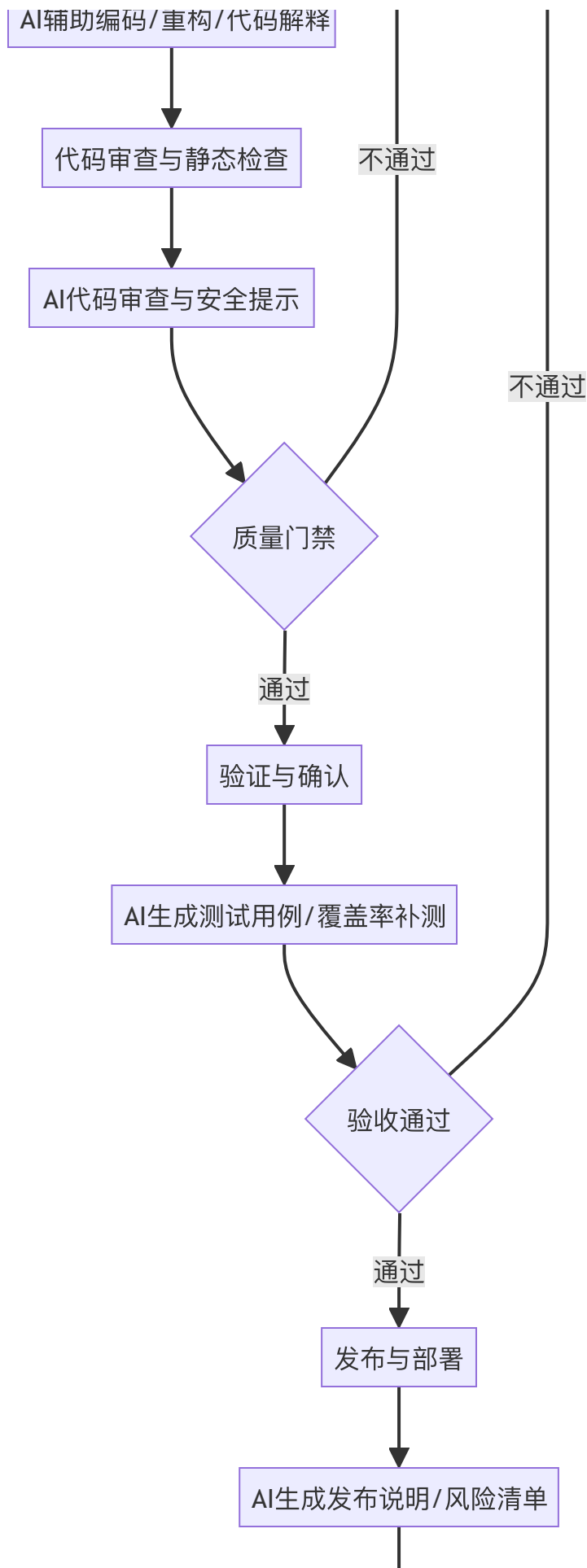
Dyad 软件过程的目标如下：

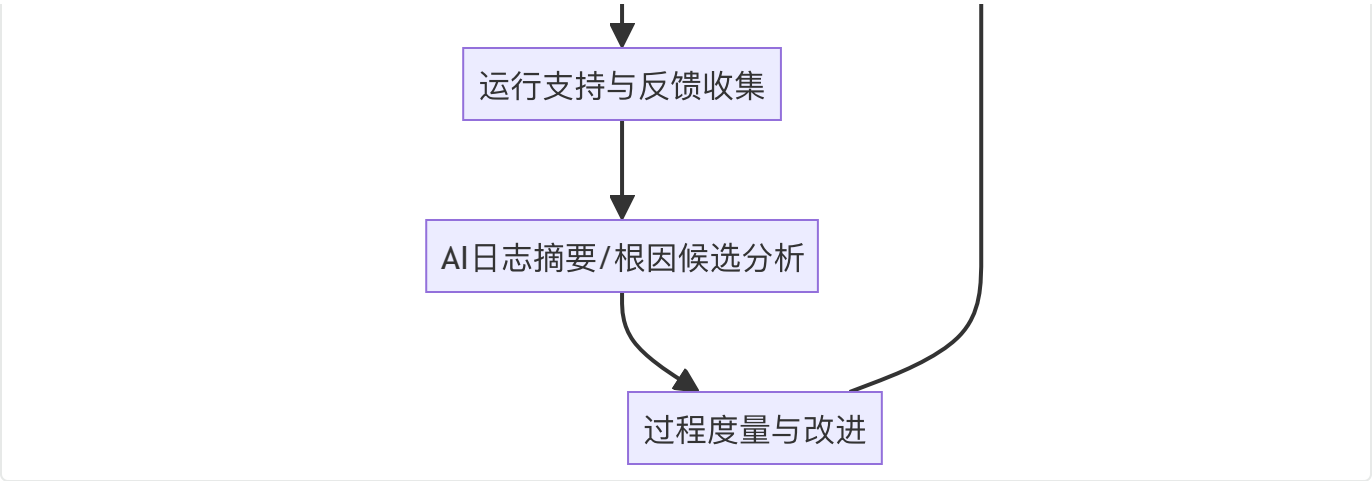
1. 明确项目要解决的问题、目标用户、功能范围和验收条件，避免需求含糊导致后续返工。
2. 建立清晰的模块边界和架构约束，使 Electron 主进程、React 渲染进程、AI 对话服务、工程生成器、配置管理、预览运行和发布模块之间职责明确。
3. 用 AI 工具辅助完成需求草案、架构说明、代码实现、代码审查、测试补充和日志分析，同时保留人工确认和自动化验证。
4. 通过类型检查、静态扫描、单元测试、集成测试、端到端测试和发布前检查，提高交付质量。
5. 记录关键工件、决策、缺陷和度量数据，让过程可以复盘和持续改进。
6. 控制 AI 引入后的风险，包括数据泄露、模型幻觉、过度依赖、错误代码生成、权限越界和测试误判。

3. 总体过程模型

Dyad 的总体过程从需求输入开始，到发布后的反馈和过程改进结束。每个阶段都允许 AI 参与，但 AI 的输出必须经过评审或验证。







总体过程采用“生成草案、人工确认、自动验证、反馈改进”的节奏。AI 可以提高速度，但不能绕过质量门禁。任何影响用户数据、本地文件系统、命令执行、依赖安装和发布打包的变更，都需要人工确认。

4. 角色职责定义

4.1 传统角色

角色	职责
项目负责人	确定迭代目标、协调任务优先级、确认发布范围和风险处理方案。
需求负责人	收集需求来源，确认用户故事、功能范围、非功能需求和验收标准。
架构负责人	维护系统架构、模块边界、接口约束和关键技术决策。
开发工程师	完成功能实现、缺陷修复、代码重构和本地调试。
测试负责人	制定测试策略，维护测试用例，确认验收结果和缺陷状态。
发布负责人	维护版本号、构建脚本、发布说明、安装包和回滚方案。
支持负责人	收集用户反馈、整理问题记录、维护常见问题和支持文档。

4.2 AI 相关角色

角色	职责	主要参与阶段
AI 过程负责人	制定 AI 工具使用规范，维护提示词模板，统计 AI 贡献度和风险事件。	全过程
AI 需求分析师	使用 AI 整理原始需求，生成需求草案、验收标准和一致性检查结果。	需求分析
AI 架构协同设计师	使用 AI 生成模块划分、UML 图、接口草案和 ADR 初稿。	系统设计
AI 代码协作者	使用 Codex、Claude Code、Cursor 等工具辅助编码、重构、解释代码和定位问题。	工程实现
AI 代码审查员	使用 AI 检查代码变更中的逻辑缺陷、安全风险、测试缺口和依赖问题。	代码审查
AI 测试分析师	使用 AI 生成测试清单、测试用例和补测建议，分析测试失败原因。	验证确认
AI 运维分析师	使用 AI 汇总日志、错误栈和用户反馈，生成根因候选和修复建议。	发布支持
AI 安全审查员	检查 AI 使用中的数据泄露、提示词注入、危险命令、权限越界和合规风险。	全过程

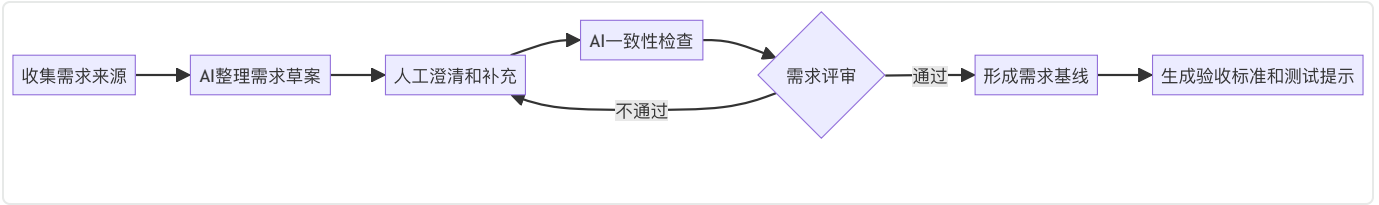
小型团队可以由同一成员兼任多个 AI 相关角色，但每个输出物仍需明确责任人。AI 工具生成的内容不能被视为无人负责的自动产物。

5. 需求分析过程

5.1 目标

需求分析过程用于明确 Dyad 的迭代目标、用户需求、功能边界、非功能约束和验收标准。需求来源包括用户反馈、GitHub issue、课堂要求、产品设想、缺陷报告和迭代复盘结论。

5.2 活动流程



5.3 主要活动

1. 收集原始需求：整理用户描述、问题反馈、作业要求、讨论记录和已有功能缺陷。
2. AI 生成需求草案：由 AI 需求分析师使用固定模板生成用户画像、用户故事、功能需求、非功能需求和验收标准。
3. 一致性检查：AI 检查需求之间的冲突、重复项、不可测试表述和范围膨胀风险。
4. 人工评审：需求负责人确认需求来源、优先级、MVP 范围和验收条件。
5. 需求基线：通过评审的需求进入迭代计划，并作为设计、实现和测试的输入。

5.4 输出物与质量标准

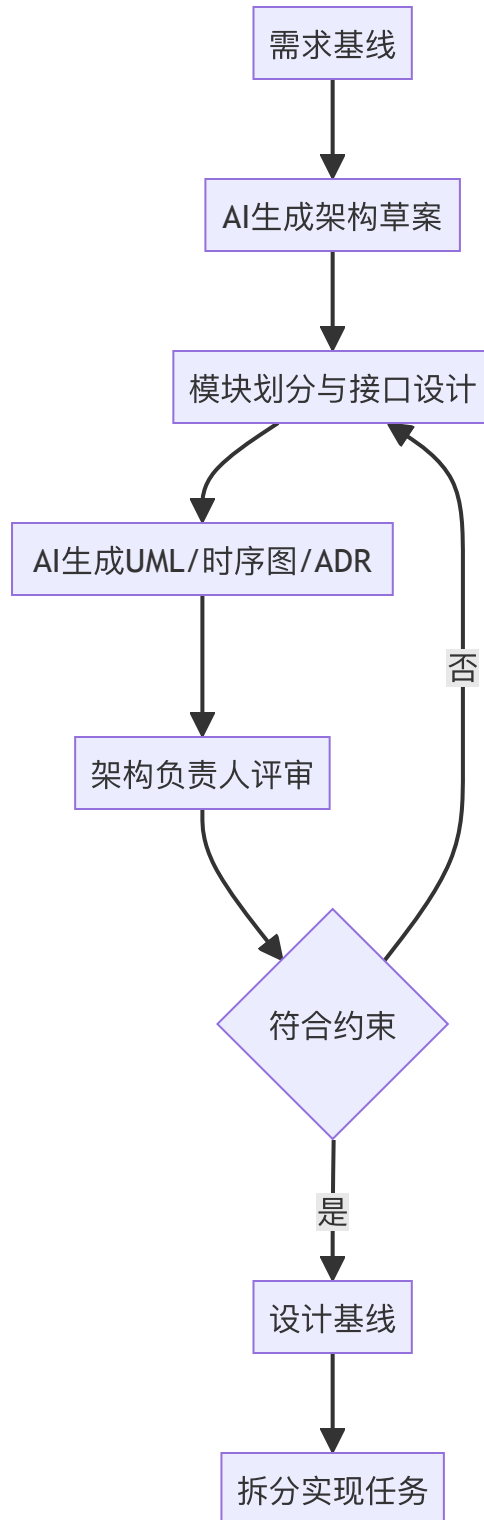
输出物	质量标准
需求列表	每条需求有编号、来源、优先级和状态。
用户故事	包含用户角色、目标和期望收益，表述不含含糊词。
非功能需求	覆盖本地运行、性能、稳定性、安全、兼容性、隐私和可维护性。
验收标准	可测试、可判断、能对应到具体功能或质量属性。
一致性检查记录	标明冲突、重复、范围风险和处理结论。

6. 系统与架构设计过程

6.1 目标

系统与架构设计过程用于把需求转换为可实现的软件结构。Dyad 的架构设计需要明确 Electron 主进程、React 渲染进程、AI 对话服务、工程生成器、本地文件操作、配置管理、预览运行、导入导出和打包发布之间的关系。

6.2 活动流程



6.3 主要活动

1. 模块划分：确定用户界面、AI 对话、提示词处理、工程生成、项目预览、配置管理和发布打包等模块。
2. 接口设计：定义主进程和渲染进程之间的 IPC 接口，明确输入输出、错误处理和权限边界。

- 3. 数据流设计：描述用户输入、模型响应、文件生成、依赖安装、预览启动和错误反馈的流转路径。
- 4. AI 生成设计草案：使用 AI 生成 UML、时序图、ADR 初稿和设计模式建议。
- 5. 架构评审：检查设计是否满足本地运行、安全边界、可测试性、扩展性和最小可行范围。

6.4 输出物与质量标准

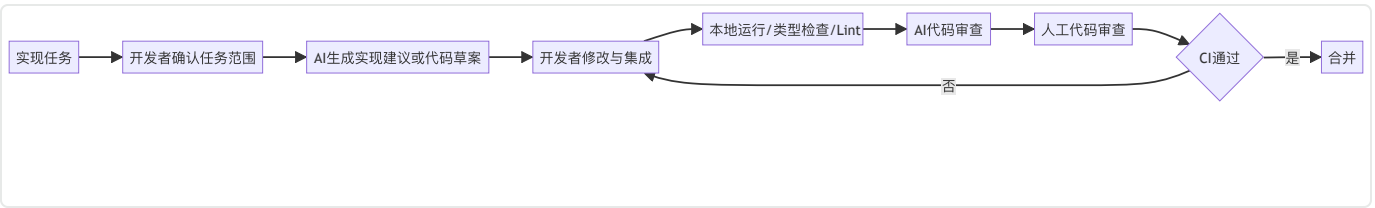
输出物	质量标准
架构说明	模块职责清楚，依赖方向明确，不出现跨层随意访问。
UML/流程图/时序图	图示与文字说明一致，能解释主要业务流。
接口定义	包含参数、返回值、异常情况、权限边界和兼容性说明。
ADR	说明背景、决策、替代方案、影响和后续约束。
任务拆分清单	每个任务可开发、可测试、可追踪到需求编号。

7. 工程实现过程

7.1 目标

工程实现过程用于将设计转化为可运行代码。Dyad 的实现过程需要保持代码可读、类型安全、模块边界清楚，并避免 AI 生成的大范围不可控变更。

7.2 活动流程



7.3 主要活动

- 1. 建立分支和任务边界：每个任务对应一个需求或缺陷，避免一次改动覆盖太多模块。
- 2. AI 辅助编码：AI 可以生成组件、工具函数、类型定义、配置逻辑、测试样例和重构建议。

3. 人工集成：开发工程师负责理解 AI 输出，调整代码风格，处理边界情况。
4. 本地验证：运行 TypeScript 类型检查、ESLint、格式化检查、单元测试和关键功能预览。
5. 代码审查：AI 先检查常见问题，人工再确认设计意图、风险和可维护性。

7.4 输出物与质量标准

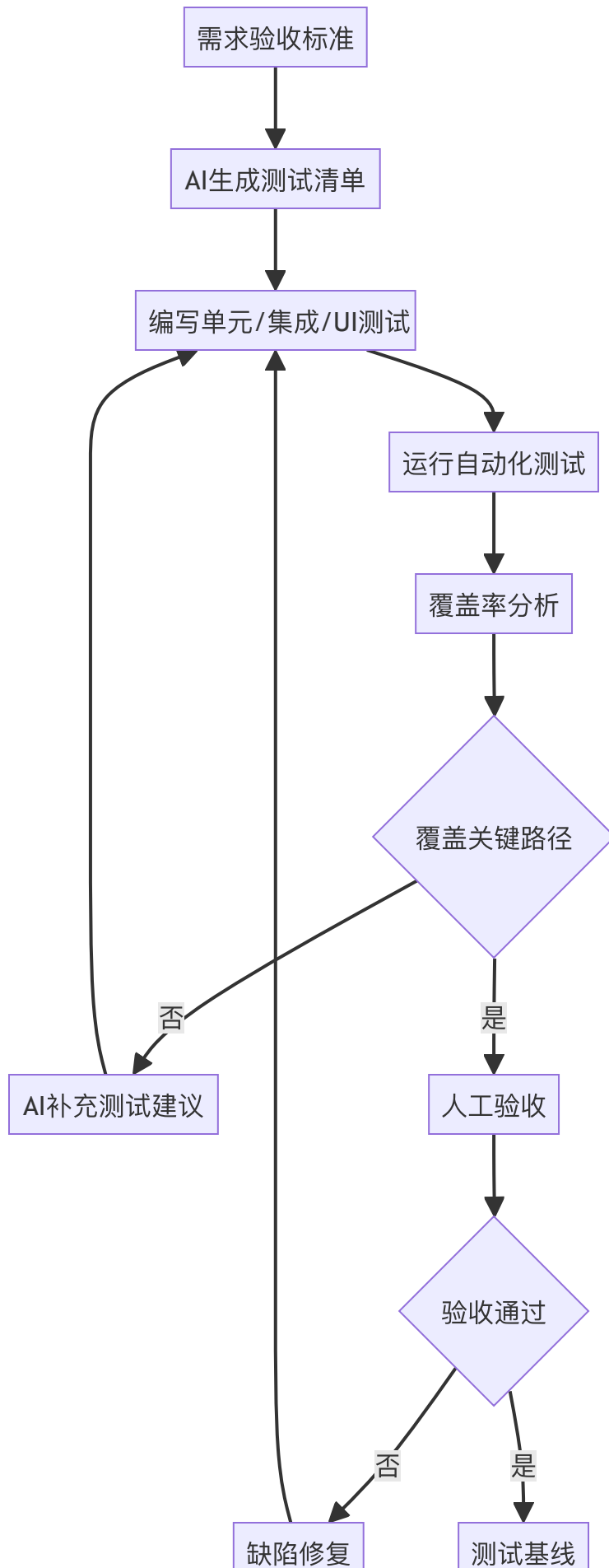
输出物	质量标准
源代码	通过类型检查，命名清晰，符合项目结构和代码风格。
AI 生成代码记录	说明生成范围、采用部分、修改内容和验证结果。
代码审查记录	包含主要风险、修改建议、测试结果和最终处理结论。
配置变更	有默认值、兼容旧配置，必要时提供迁移说明。
安全检查记录	文件写入、命令执行、依赖安装和模型输出处理经过检查。

8. 验证与确认过程

8.1 目标

验证与确认过程用于确认软件是否满足需求和质量要求。Dyad 需要重点验证自然语言生成应用、增量修改、本地预览、配置保存、跨平台路径、文件系统访问和发布准备等核心流程。

8.2 活动流程



8.3 主要活动

- 1. 测试计划制定：根据需求验收标准确定单元测试、集成测试、端到端测试和手工验证范围。
- 2. AI 生成测试清单：AI 测试分析师根据需求和代码生成测试候选项。
- 3. 自动化测试实现：开发者和测试负责人将测试候选项转化为 Vitest、Jest 或 Playwright 测试。
- 4. 覆盖率检查：检查核心模块和关键路径是否被覆盖。
- 5. 自愈测试分析：当 UI 测试失败时，AI 可以分析失败原因，但脚本修复必须经测试负责人确认。
- 6. 验收确认：需求负责人和测试负责人共同确认功能是否达到验收标准。

8.4 输出物与质量标准

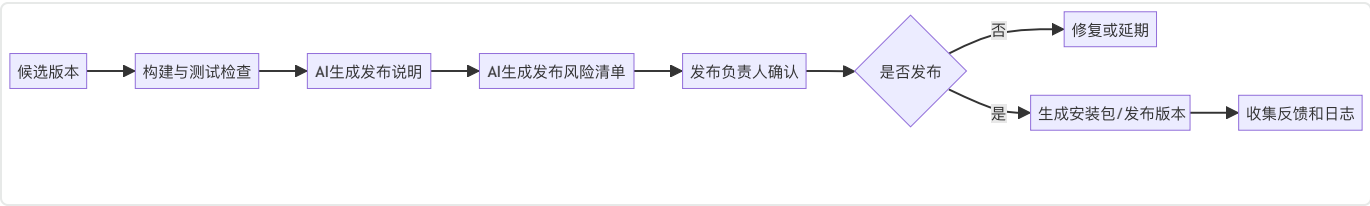
输出物	质量标准
测试计划	覆盖核心流程、异常路径、兼容性和安全边界。
测试用例	有输入、步骤、预期结果和关联需求编号。
自动化测试脚本	可重复运行，不依赖不稳定外部环境。
覆盖率报告	标明未覆盖分支和补测计划。
缺陷记录	包含复现步骤、环境信息、严重程度、修复版本和验证结果。

9. 发布与部署过程

9.1 目标

发布与部署过程用于将软件从开发状态转为可交付状态。Dyad 的发布活动包括版本号确认、构建检查、安装包生成、发布说明、变更记录、下载方式和回滚方案。

9.2 活动流程



9.3 主要活动

- 1. 发布准备：确认本轮需求、缺陷、测试和文档状态。
- 2. 构建检查：执行 lint、类型检查、自动化测试和安装包构建。
- 3. AI 生成发布说明：根据提交记录、PR、issue 和变更说明生成初稿。
- 4. 发布风险评估：AI 辅助列出高风险变更、兼容性影响和回滚建议。
- 5. 人工确认发布：发布负责人确认版本号、发布说明、已知问题和回滚路径。
- 6. 发布后监控：收集用户反馈、错误日志和安装问题。

9.4 输出物与质量标准

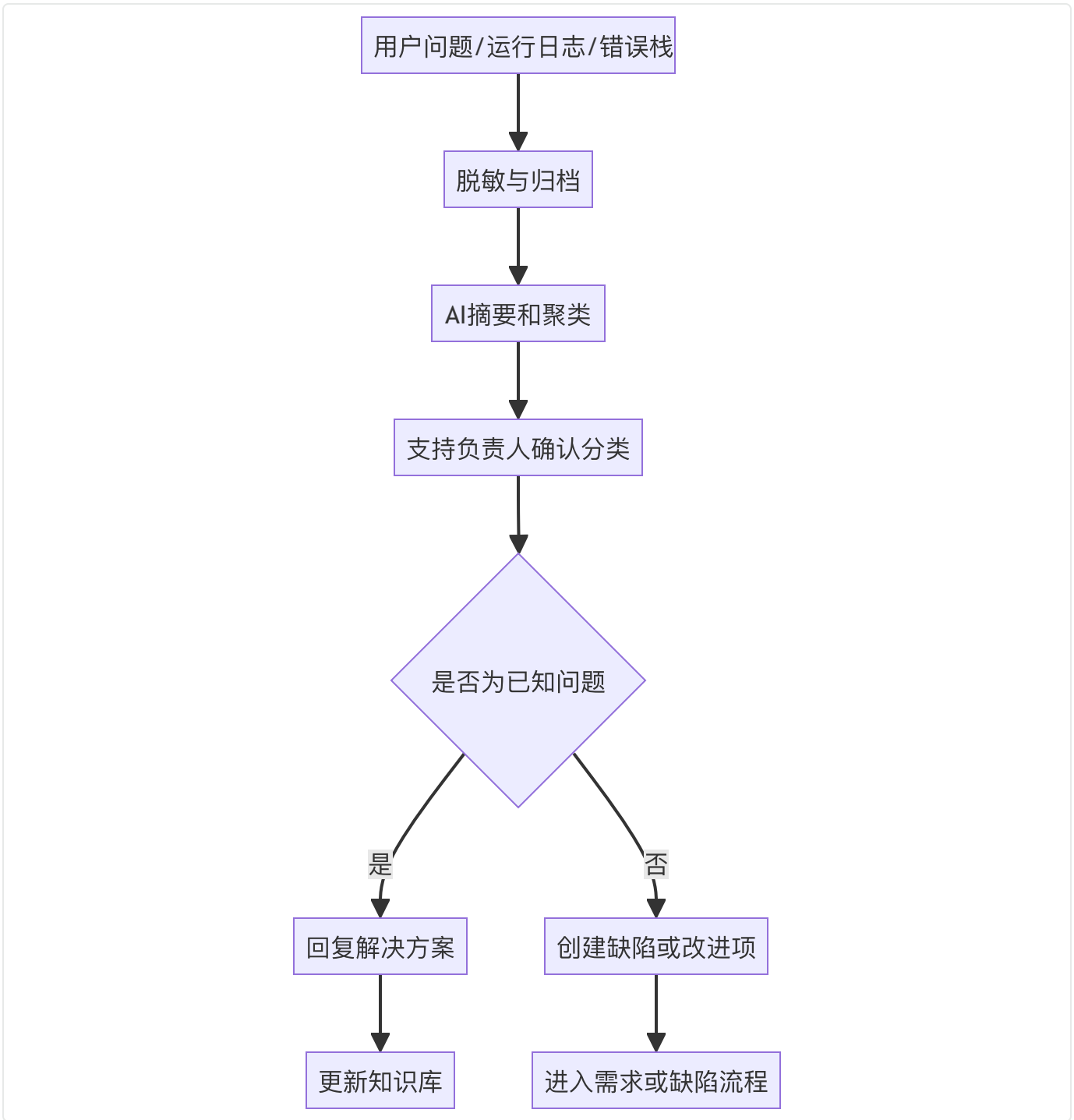
输出物	质量标准
发布说明	包含新增功能、修复问题、已知问题和兼容性说明。
构建产物	可安装、可启动、关键流程可运行。
发布风险清单	标明风险等级、影响范围和回滚方式。
版本记录	与代码提交、issue 和测试结果对应。
回滚方案	明确触发条件、操作步骤和负责人。

10. 配置与支持过程

10.1 目标

配置与支持过程用于维护代码版本、项目配置、文档、问题跟踪和用户支持。Dyad 作为本地应用，尤其需要管理不同操作系统、模型配置、项目路径、依赖版本和本地权限差异。

10.2 活动流程



10.3 主要活动

1. 版本管理：使用 Git 管理代码、文档、配置和发布标签。
2. 配置管理：记录模型配置、项目路径、模板版本、依赖版本和默认设置。
3. 问题跟踪：对用户反馈和缺陷进行分类、优先级评估和处理状态维护。
4. AI 日志分析：对脱敏后的日志、错误栈和反馈描述进行摘要、聚类和根因候选分析。
5. 文档维护：更新 README、安装说明、常见问题、发布说明和开发者文档。

10.4 输出物与质量标准

输出物	质量标准
配置项记录	版本、用途、默认值和影响范围明确。
问题记录	有环境信息、复现步骤、日志摘要和处理状态。
日志分析结果	已脱敏，有证据链，不把猜测当结论。
知识库条目	包含现象、原因、解决步骤和适用版本。
文档更新	与当前功能一致，避免过期说明。

11. 项目管理过程

11.1 目标

项目管理过程用于控制范围、时间、质量和风险。Dyad 的迭代周期应围绕 MVP 和真实反馈展开，避免功能扩张过快。

11.2 主要活动

- 1. 制定迭代目标：确定本轮要完成的需求、缺陷和改进项。
- 2. 任务拆分：将需求拆分为可开发、可测试、可验收的任务。
- 3. 进度跟踪：记录任务状态、阻塞问题和变更原因。
- 4. 风险管理：识别模型调用不稳定、跨平台兼容、安装复杂度、提示词注入、文件写入风险等问题。
- 5. AI 贡献度统计：记录 AI 在需求、设计、编码、测试和支持中的参与情况。

11.3 项目管理流程



12. 过程改进过程

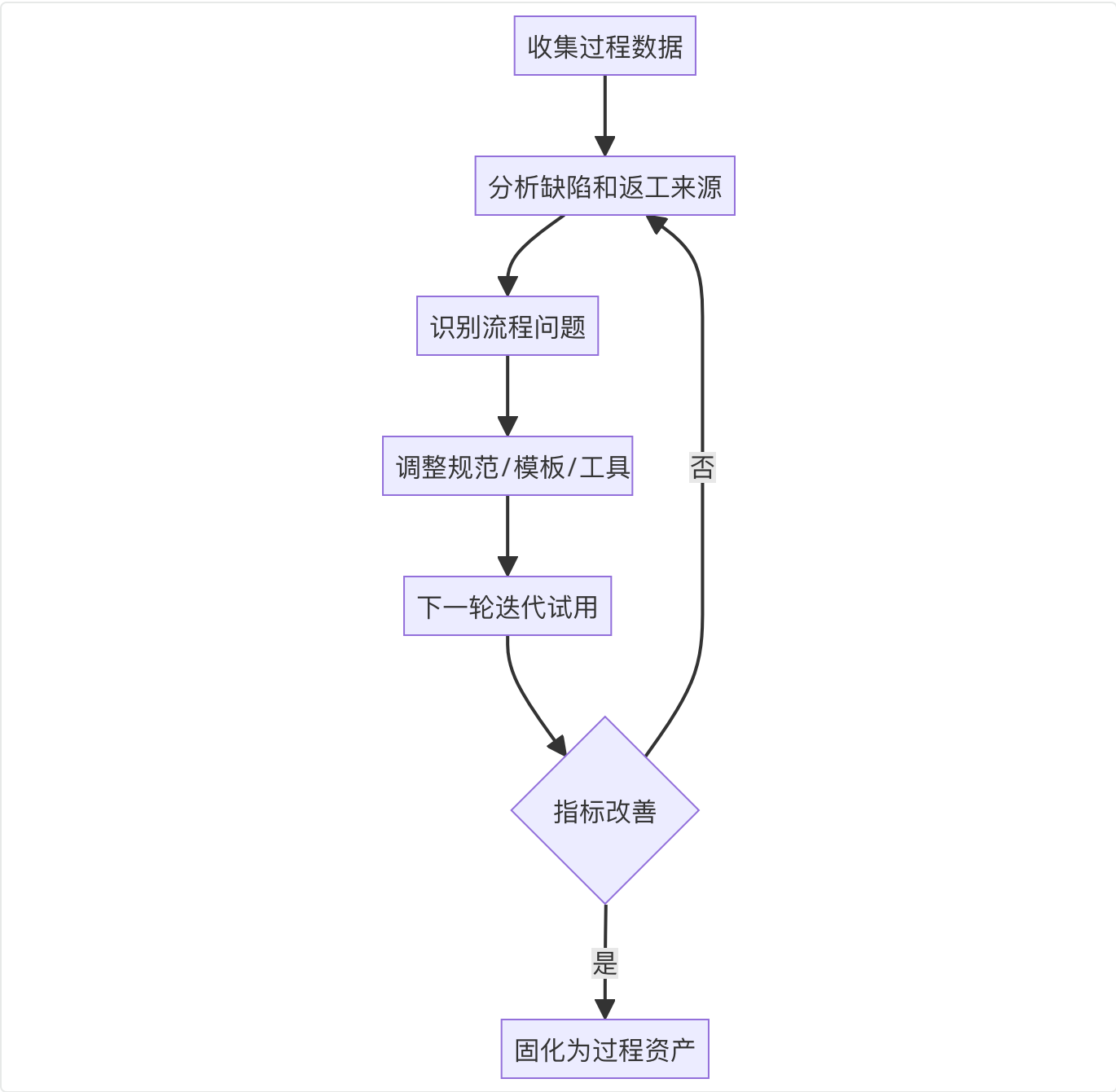
12.1 目标

过程改进过程用于根据真实开发数据优化 Dyad 的工作方式。复盘重点包括需求返工、缺陷来源、测试失败、发布问题、AI 输出采纳率和 AI 风险事件。

12.2 主要活动

1. 收集度量数据：统计需求处理时间、代码审查时间、测试覆盖率、缺陷修复时间、发布问题数量和 AI 输出采纳率。
2. 分析问题来源：区分需求不清、设计遗漏、实现缺陷、测试不足、环境差异和 AI 输出错误。
3. 调整流程：更新提示词模板、评审清单、测试策略、发布检查清单和安全规则。
4. 沉淀经验：把有效做法写入过程规范，把高频问题写入知识库。
5. 验证改进效果：在后续迭代中检查指标是否改善。

12.3 改进流程



13. 交付物清单与质量标准

阶段	交付物	质量标准
需求分析	需求列表、用户故事、验收标准、一致性检查记录	可追踪、可测试、范围清楚、冲突已处理。
架构设计	架构说明、UML/流程图、接口定义、ADR	模块边界清楚，接口可实现，设计决策有依据。

工程实现	源代码、配置文件、AI 生成记录、代码审查记录	通过类型检查和静态检查，符合项目风格。
测试验证	测试计划、测试用例、自动化脚本、覆盖率报告、缺陷记录	覆盖核心路径和异常路径，测试可重复执行。
发布部署	发布说明、构建产物、版本记录、风险清单、回滚方案	构建可用，说明清楚，风险和回滚路径明确。
支持维护	问题记录、日志分析、知识库、文档更新	信息已脱敏，结论有证据，解决步骤可执行。
过程改进	度量报告、复盘记录、流程调整项	数据真实，问题归因清楚，改进项可跟踪。

14. 质量门禁

门禁点	检查内容	通过条件
需求门禁	需求来源、范围、验收标准、一致性检查	需求负责人确认，验收标准可测试。
设计门禁	模块边界、接口、数据流、安全约束	架构负责人确认，无明显跨层依赖。
代码门禁	类型检查、Lint、单元测试、代码审查	CI 通过，核心变更有人审查。
测试门禁	自动化测试、覆盖率、缺陷状态	核心流程通过，严重缺陷关闭或延期说明明确。
发布门禁	构建结果、发布说明、风险清单、回滚方案	发布负责人确认，回滚路径可执行。
AI 安全门禁	数据脱敏、权限边界、危险命令、AI 输出审查	无敏感数据泄露，无未确认的破坏性操作。

15. 过程度量指标

15.1 效率指标

指标	说明	目标
需求初稿完成时间	从需求输入到形成初稿的时间	持续下降，AI 辅助后减少 30% 以上。
需求评审返工次数	需求因表达不清被退回的次数	每轮迭代不超过 2 次。
平均 PR 审查时间	从提交 PR 到审查完成的时间	常规变更控制在 1 个工作日内。
缺陷平均修复时间	从缺陷确认到修复合并的时间	按严重程度设定目标。
发布准备时间	从候选版本到发布完成的时间	逐轮减少重复整理工作。

15.2 质量指标

指标	说明	目标
自动化测试通过率	CI 中自动化测试通过比例	主分支保持 95% 以上。
核心模块测试覆盖率	工程生成、配置管理、预览运行等模块覆盖率	稳定达到 70% 以上。
缺陷逃逸率	发布后发现缺陷数 / 总缺陷数	持续下降。
严重缺陷数	阻塞发布或影响数据安全的缺陷数量	发布前必须为 0。
发布回滚次数	发布后因严重问题回滚的次数	作为发布质量复盘重点。

15.3 AI 贡献度指标

指标	说明	使用方式
AI 生成需求采纳率	被采纳的 AI 需求条目 / AI 生成需求条目	判断需求提示词是否有效。
AI 代码采纳率	合并代码中由 AI 生成或辅助修改的比例	评估 AI 对实现阶段的贡献。

AI 审查有效问题数	AI 审查中被确认的问题数量	判断 AI 审查价值。
AI 测试用例采纳率	被保留并运行的 AI 生成测试数 / AI 生成测试数	评估 AI 测试生成质量。
AI 日志分析命中率	AI 根因候选被人工确认的比例	评估运维分析可靠性。
AI 输出返工率	AI 输出被大幅修改或废弃的比例	返工率过高时调整提示词或工具。
AI 风险事件数	数据泄露、错误建议、危险操作等事件数量	作为安全审查和过程改进依据。

AI 贡献度指标不能单独作为绩效评价依据。它用于判断工具是否真正节省时间、提高质量，以及哪些环节需要调整提示词、模型或流程。

16. AI 使用规范

- 1. 输入 AI 工具前应检查是否包含密钥、用户隐私、未公开数据和敏感路径。
- 2. 涉及用户数据、文件系统、命令执行、依赖安装和发布操作的 AI 建议必须人工确认。
- 3. AI 生成代码必须经过类型检查、静态检查、测试和人工审查。
- 4. AI 生成测试必须检查断言质量，避免只覆盖实现细节。
- 5. AI 日志分析必须使用脱敏数据，根因结论要有日志、复现步骤或代码变更作为证据。
- 6. AI 工具的提示词模板、重要输出和采纳情况应保存到项目过程记录中。
- 7. 当 AI 输出与项目约束冲突时，以项目过程定义、需求基线和人工评审结论为准。

17. 适用与裁剪规则

Dyad 的过程可以按任务风险进行裁剪：

任务类型	过程要求
文档修正、轻量 UI 文案修改	可简化设计评审，但需要基本 review。
普通功能开发	需要需求确认、实现、测试和 PR 审查。
核心生成器、模型调用、文件系统访问	需要完整需求、架构评审、安全检查和回归测试。
发布打包、配置迁移	需要发布风险清单、回滚方案和跨平台验证。

探索性原型	可先快速验证，但不得直接进入正式发布。
-------	---------------------

过程裁剪由项目负责人确认。裁剪只能减少低风险任务的文档和会议成本，不能取消安全、测试和发布门禁。

18. 过程总结

Dyad 软件过程以轻量迭代为主线，以 AI 辅助为加速手段，以人工确认和自动化验证控制风险。需求阶段用 AI 提高规格整理速度，设计阶段用 AI 辅助模块划分和图示表达，实现阶段用 AI 支持编码和审查，测试阶段用 AI 补充用例和分析失败，发布支持阶段用 AI 汇总日志和生成根因候选。

这个过程的核心要求很简单：每个阶段都要有清楚的输入、输出和质量标准；AI 可以参与生成和分析，但所有关键决策都要可追踪、可验证、有人负责。这样既能保持 Dyad 快速迭代的特点，也能让需求、代码、测试和发布质量逐步稳定下来。